MongoDB常见面试题总结（下）

Guide 数据库 NoSQL, MongoDB 约 4997 字 大约 17 分钟

MongoDB 索引

MongoDB 索引有什么用?

和关系型数据库类似，MongoDB 中也有索引。索引的目的主要是用来提高查询效率，如果没有索引的话，MongoDB 必须执行 **集合扫描** ，即扫描集合中的每个文档，以选择与查询语句匹配的文档。如果查询存在合适的索引，MongoDB 可以使用该索引来限制它必须检查的文档数量。并且，MongoDB 可以使用索引中的排序返回排序后的结果。

虽然索引可以显著缩短查询时间，但是使用索引、维护索引是有代价的。在执行写入操作时，除了要更新文档之外，还必须更新索引，这必然会影响写入的性能。因此，当有大量写操作而读操作少时，或者不考虑读操作的性能时，都不推荐建立索引。

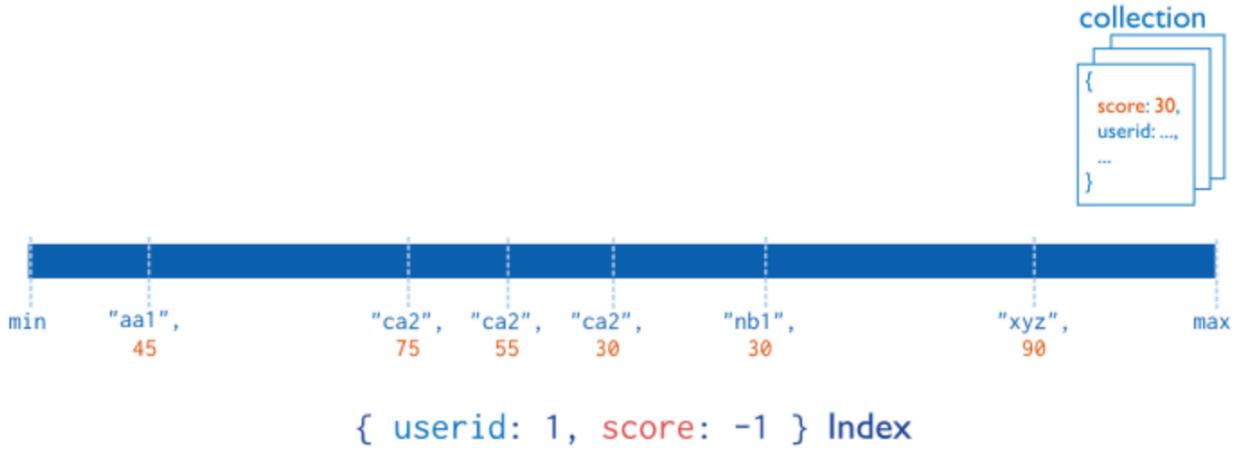
MongoDB 支持哪些类型的索引?

MongoDB 支持多种类型的索引，包括单字段索引、复合索引、多键索引、哈希索引、文本索引、 地理位置索引等，每种类型的索引有不同的使用场合。

- **单字段索引：** 建立在单个字段上的索引，索引创建的排序顺序无所谓，MongoDB 可以头/尾开始遍历。
- **复合索引：** 建立在多个字段上的索引，也可以称之为组合索引、联合索引。
- **多键索引**：MongoDB 的一个字段可能是数组，在对这种字段创建索引时，就是多键索引。MongoDB 会为数组的每个值创建索引。就是说你可以按照数组里面的值做条件来查询，这个时候依然会走索引。
- **哈希索引**：按数据的哈希值索引，用在哈希分片集群上。
- **文本索引：** 支持对字符串内容的文本搜索查询。文本索引可以包含任何值为字符串或字符串元素数组的字段。一个集合只能有一个文本搜索索引，但该索引可以覆盖多个字段。MongoDB 虽然支持全文索引，但是性能低下，暂时不建议使用。
- **地理位置索引：** 基于经纬度的索引，适合 2D 和 3D 的位置查询。
- **唯一索引**：确保索引字段不会存储重复值。如果集合已经存在了违反索引的唯一约束的文档，则后台创建唯一索引会失败。
- **TTL 索引**：TTL 索引提供了一个过期机制，允许为每一个文档设置一个过期时间，当一个文档达到预设的过期时间之后就会被删除。
- ……

复合索引中字段的顺序有影响吗？

复合索引中字段的顺序非常重要，例如下图中的复合索引由 `{userid:1, score:-1}` 组成，则该复合索引首先按照 `userid` 升序排序；然后再每个 `userid` 的值内，再按照 `score` 降序排序。



在复合索引中，按照何种方式排序，决定了该索引在查询中是否能被应用到。

走复合索引的排序：

```sql
db.s2.find().sort({"userid": 1, "score": -1})
db.s2.find().sort({"userid": -1, "score": 1})
```

不走复合索引的排序：

```sql
db.s2.find().sort({"userid": 1, "score": 1})
db.s2.find().sort({"userid": -1, "score": -1})
db.s2.find().sort({"score": 1, "userid": -1})
db.s2.find().sort({"score": 1, "userid": 1})
db.s2.find().sort({"score": -1, "userid": -1})
db.s2.find().sort({"score": -1, "userid": 1})
```

我们可以通过 explain 进行分析：

```sql
db.s2.find().sort({"score": -1, "userid": 1}).explain()
```

复合索引遵循左前缀原则吗?

MongoDB 的复合索引遵循左前缀原则：拥有多个键的索引，可以同时得到所有这些键的前缀组成的索引，但不包括除左前缀之外的其他子集。比如说，有一个类似 `{a: 1, b: 1, c: 1, ..., z: 1}` 这样的索引，那么实际上也等于有了 `{a: 1}` 、 `{a: 1, b: 1}` 、 `{a: 1, b: 1, c: 1}` 等一系列索引，但是不会有 `{b: 1}` 这样的非左前缀的索引。

什么是 TTL 索引?

TTL 索引提供了一个过期机制，允许为每一个文档设置一个过期时间 `expireAfterSeconds` ，当一个文档达到预设的过期时间之后就会被删除。TTL 索引除了有 `expireAfterSeconds` 属性外，和普通索引一样。

数据过期对于某些类型的信息很有用，比如机器生成的事件数据、日志和会话信息，这些信息只需要在数据库中保存有限的时间。

TTL 索引运行原理：

- MongoDB 会开启一个后台线程读取该 TTL 索引的值来判断文档是否过期，但不会保证已过期的数据会立马被删除，因后台线程每 60 秒触发一次删除任务，且如果删除的数据量较大，会存在上一次的删除未完成，而下一次的任务已经开启的情况，导致过期的数据也会出现超过了数据保留时间 60 秒以上的现象。
- 对于副本集而言，TTL 索引的后台进程只会在 Primary 节点开启，在从节点会始终处于空闲状态，从节点的数据删除是由主库删除后产生的 oplog 来做同步。

TTL 索引限制：

- TTL 索引是单字段索引。复合索引不支持 TTL
- `_id` 字段不支持 TTL 索引。
- 无法在上限集合(Capped Collection)上创建 TTL 索引，因为 MongoDB 无法从上限集合中删除文档。
- 如果某个字段已经存在非 TTL 索引，那么在该字段上无法再创建 TTL 索引。

什么是覆盖索引查询?

根据官方文档介绍，覆盖查询是以下的查询：

- 所有的查询字段是索引的一部分。

- 结果中返回的所有字段都在同一索引中。
- 查询中没有字段等于 `null` 。

由于所有出现在查询中的字段是索引的一部分， MongoDB 无需在整个数据文档中检索匹配查询条件和返回使用相同索引的查询结果。因为索引存在于内存中，从索引中获取数据比通过扫描文档读取数据要快得多。

举个例子：我们有如下 `users` 集合:

```json
{
   "_id": ObjectId("53402597d852426020000002"),
   "contact": "987654321",
   "dob": "01-01-1991",
   "gender": "M",
   "name": "Tom Benzamin",
   "user_name": "tombenzamin"
}
```

我们在 `users` 集合中创建联合索引，字段为 `gender` 和 `user_name` :

```sql
db.users.ensureIndex({gender:1,user_name:1})
```

现在，该索引会覆盖以下查询：

```sql
db.users.find({gender:"M"},{user_name:1,_id:0})
```

为了让指定的索引覆盖查询，必须显式地指定 `_id: 0` 来从结果中排除 `_id` 字段，因为索引不包括 `_id` 字段。

MongoDB 高可用

复制集群

什么是复制集群?

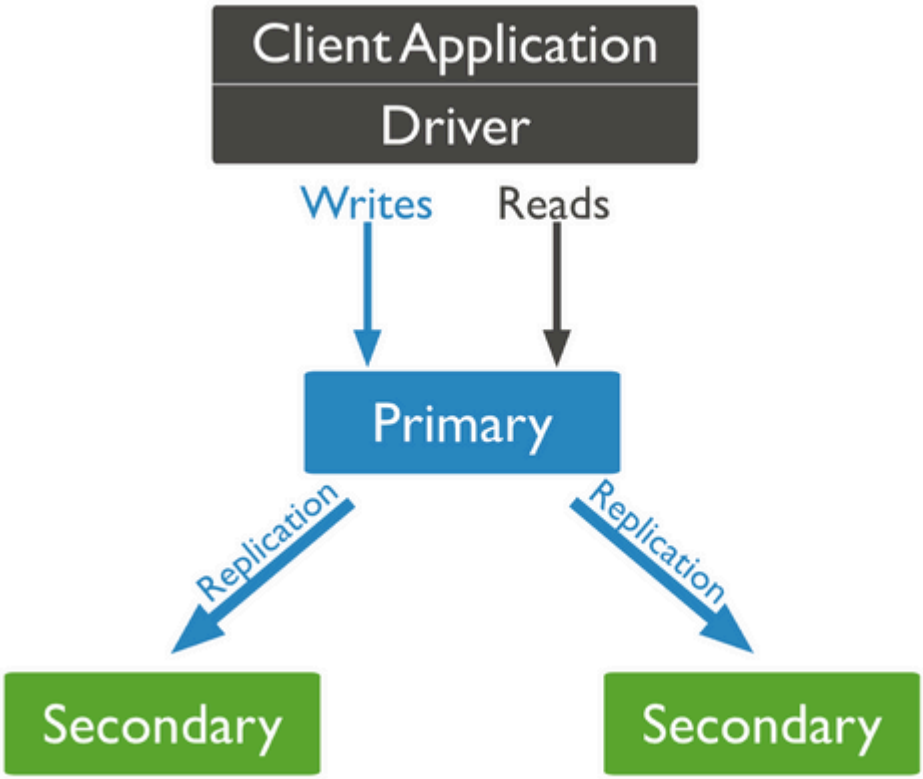
MongoDB 的复制集群又称为副本集群，是一组维护相同数据集合的 mongod 进程。

客户端连接到整个 Mongodb 复制集群，主节点机负责整个复制集群的写，从节点可以进行读操作，但默认还是主节点负责整个复制集群的读。主节点发生故障时，自动从从节点中选举出一个新的主节点，确保集群的正常使用，这对于客户端来说是无感知的。

通常来说，一个复制集群包含 1 个主节点（Primary），多个从节点（Secondary）以及零个或 1 个仲裁节点（Arbiter）。

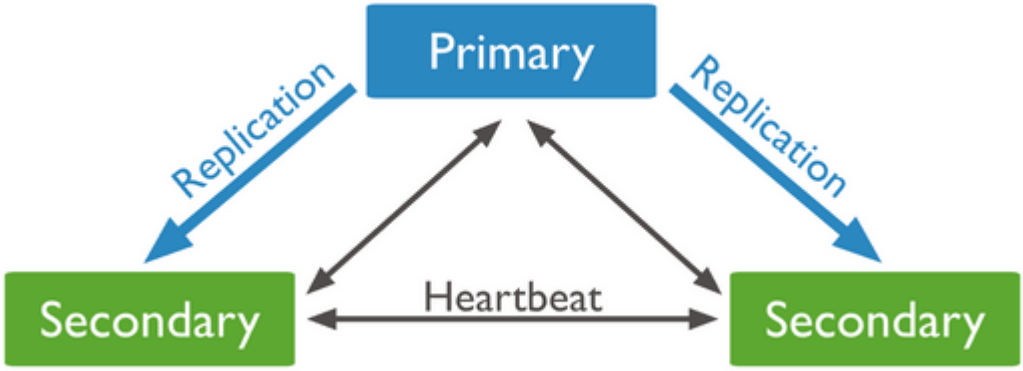
- **主节点**：整个集群的写操作入口，接收所有的写操作，并将集合所有的变化记录到操作日志中，即 oplog。主节点挂掉之后会自动选出新的主节点。
- **从节点**：从主节点同步数据，在主节点挂掉之后选举新节点。不过，从节点可以配置成 0 优先级，阻止它在选举中成为主节点。
- **仲裁节点**：这个是为了节约资源或者多机房容灾用，只负责主节点选举时投票不存数据，保证能有节点获得多数赞成票。

下图是一个典型的三成员副本集群：



主节点与备节点之间是通过 **oplog（操作日志）** 来同步数据的。oplog 是 local 库下的一个特殊的 **上限集合(Capped Collection)**，用来保存写操作所产生的增量日志，类似于 MySQL 中 的 Binlog。

> 上限集合类似于定长的循环队列，数据顺序追加到集合的尾部，当集合空间达到上限时，它会覆盖集合中最旧的文档。上限集合的数据将会被顺序写入到磁盘的固定空间内，所以，I/O 速度非常快，如果不建立索引，性能更好。



点击放大

当主节点上的一个写操作完成后，会向 oplog 集合写入一条对应的日志，而从节点则通过这个 oplog 不断拉取到新的日志，在本地进行回放以达到数据同步的目的。

副本集最多有一个主节点。 如果当前主节点不可用，一个选举会抉择出新的主节点。MongoDB 的节点选举规则能够保证在 Primary 挂掉之后选取的新节点一定是集群中数据最全的一个。

为什么要用复制集群?

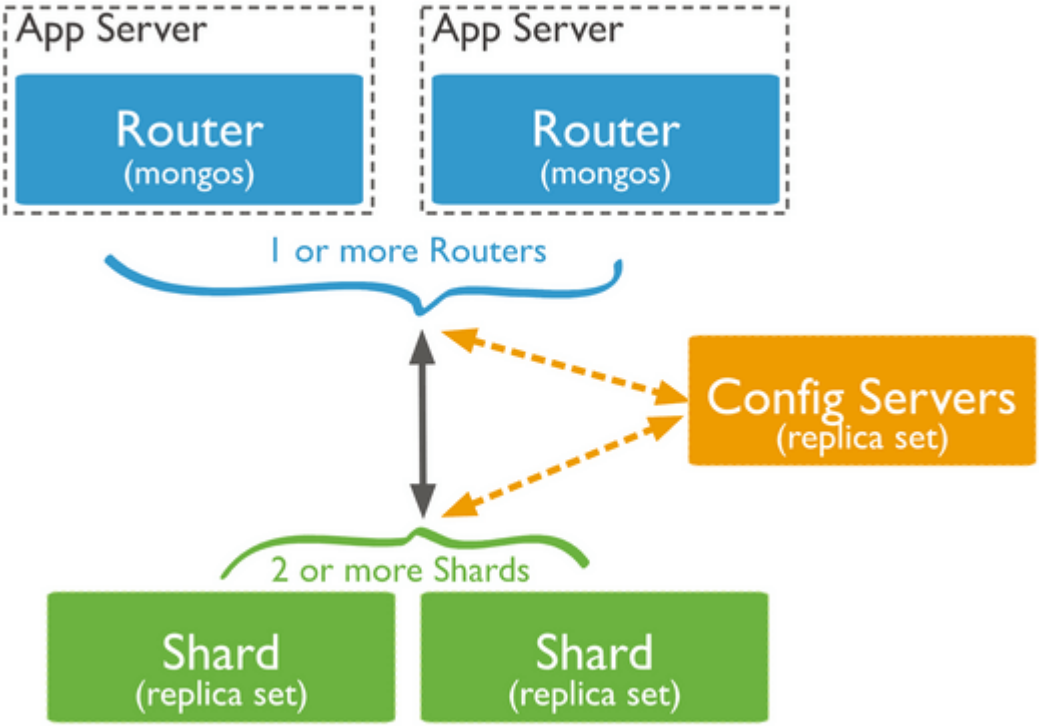
- **实现 failover**：提供自动故障恢复的功能，主节点发生故障时，自动从从节点中选举出一个新的主节点，确保集群的正常使用，这对于客户端来说是无感知的。
- **实现读写分离**：我们可以设置从节点上可以读取数据，主节点负责写入数据，这样的话就实现了读写分离，减轻了主节点读写压力过大的问题。MongoDB 4.0 之前版本如果主库压力不大,不建议读写分离，因为写会阻塞读，除非业务对响应时间不是非常关注以及读取历史数据接受一定时间延迟。

分片集群

什么是分片集群?

分片集群是 MongoDB 的分布式版本，相较副本集，分片集群数据被均衡的分布在不同分片中， 不仅大幅提升了整个集群的数据容量上限，也将读写的压力分散到不同分片，以解决副本集性能瓶颈的难题。

MongoDB 的分片集群由如下三个部分组成（下图来源于官方文档对分片集群的介绍 ）：



- **Config Servers**：配置服务器，本质上是一个 MongoDB 的副本集，负责存储集群的各种元数据和配置，如分片地址、Chunks 等
- **Mongos**：路由服务，不存具体数据，从 Config 获取集群配置讲请求转发到特定的分片，并且整合分片结果返回给客户端。
- **Shard**：每个分片是整体数据的一部分子集，从 MongoDB3.6 版本开始，每个 Shard 必须部署为副本集（replica set）架构

为什么要用分片集群?

随着系统数据量以及吞吐量的增长，常见的解决办法有两种：垂直扩展和水平扩展。

垂直扩展通过增加单个服务器的能力来实现，比如磁盘空间、内存容量、CPU 数量等；水平扩展则通过将数据存储到多个服务器上来实现，根据需要添加额外的服务器以增加容量。

类似于 Redis Cluster，MongoDB 也可以通过分片实现 **水平扩展** 。水平扩展这种方式更灵活，可以满足更大数据量的存储需求，支持更高吞吐量。并且，水平扩展所需的整体成本更低，仅仅需要相对较低配置的单机服务器即可，代价是增加了部署的基础设施和维护的复杂性。

也就是说当你遇到如下问题时，可以使用分片集群解决：

- 存储容量受单机限制，即磁盘资源遭遇瓶颈。
- 读写能力受单机限制，可能是 CPU、内存或者网卡等资源遭遇瓶颈，导致读写能力无法扩展。

什么是分片键?

分片键（Shard Key） 是数据分区的前提， 从而实现数据分发到不同服务器上，减轻服务器的负担。也就是说，分片键决定了集合内的文档如何在集群的多个分片间的分布状况。

分片键就是文档里面的一个字段，但是这个字段不是普通的字段，有一定的要求：

- 它必须在所有文档中都出现。
- 它必须是集合的一个索引，可以是单索引或复合索引的前缀索引，不能是多索引、文本索引或地理空间位置索引。
- MongoDB 4.2 之前的版本，文档的分片键字段值不可变。MongoDB 4.2 版本开始，除非分片键字段是不可变的 `_id` 字段，否则您可以更新文档的分片键值。MongoDB 5.0 版本开始，实现了实时重新分片（live resharding），可以实现分片键的完全重新选择。
- 它的大小不能超过 512 字节。

如何选择分片键?

选择合适的片键对 sharding 效率影响很大，主要基于如下四个因素（摘自分片集群使用注意事项 - - 腾讯云文档 ）：

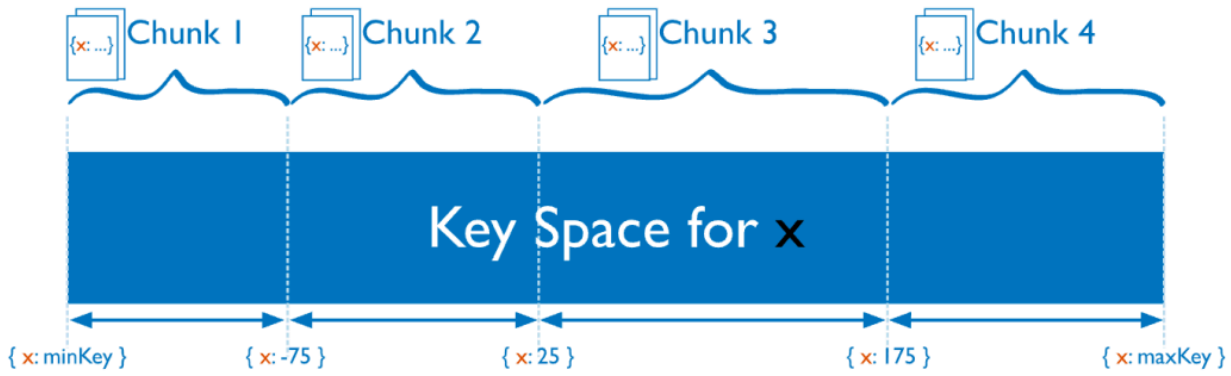
- **取值基数** 取值基数建议尽可能大，如果用小基数的片键，因为备选值有限，那么块的总数量就有限，随着数据增多，块的大小会越来越大，导致水平扩展时移动块会非常困难。 例如：选择年龄做一个基数，范围最多只有 100 个，随着数据量增多，同一个值分布过多时，导致 chunck 的增长超出 chuncksize 的范围，引起 jumbo chunk，从而无法迁移，导致数据分布不均匀，性能瓶颈。
- **取值分布** 取值分布建议尽量均匀，分布不均匀的片键会造成某些块的数据量非常大，同样有上面数据分布不均匀，性能瓶颈的问题。
- **查询带分片** 查询时建议带上分片，使用分片键进行条件查询时，mongos 可以直接定位到具体分片，否则 mongos 需要将查询分发到所有分片，再等待响应返回。
- **避免单调递增或递减** 单调递增的 sharding key，数据文件挪动小，但写入会集中，导致最后一篇的数据量持续增大，不断发生迁移，递减同理。

综上，在选择片键时要考虑以上 4 个条件，尽可能满足更多的条件，才能降低 MoveChunks 对性能的影响，从而获得最优的性能体验。

分片策略有哪些?

MongoDB 支持两种分片算法来满足不同的查询需求（摘自MongoDB 分片集群介绍 - 阿里云文档 ）：

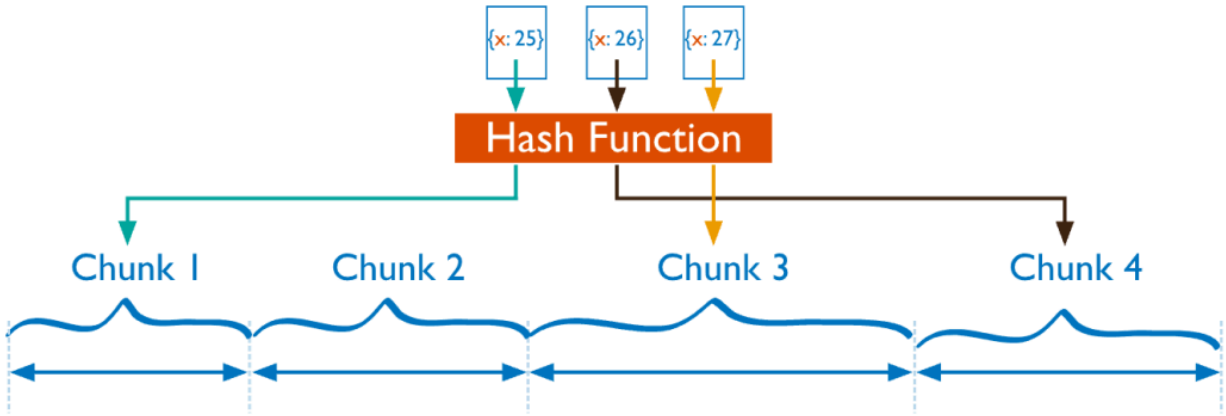
1、基于范围的分片：



MongoDB 按照分片键（Shard Key）的值的范围将数据拆分为不同的块（Chunk），每个块包含了一段范围内的数据。当分片键的基数大、频率低且值非单调变更时，范围分片更高效。

- 优点：Mongos 可以快速定位请求需要的数据，并将请求转发到相应的 Shard 节点中。
- 缺点：可能导致数据在 Shard 节点上分布不均衡，容易造成读写热点，且不具备写分散性。
- 适用场景：分片键的值不是单调递增或单调递减、分片键的值基数大且重复的频率低、需要范围查询等业务场景。

2、基于 Hash 值的分片



MongoDB 计算单个字段的哈希值作为索引值，并以哈希值的范围将数据拆分为不同的块（Chunk）。

- 优点：可以将数据更加均衡地分布在各 Shard 节点中，具备写分散性。

- 缺点：不适合进行范围查询，进行范围查询时，需要将读请求分发到所有的 Shard 节点。
- 适用场景：分片键的值存在单调递增或递减、片键的值基数大且重复的频率低、需要写入的数据随机分发、数据读取随机性较大等业务场景。

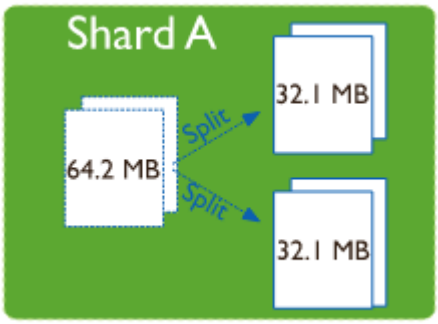
除了上述两种分片策略，您还可以配置 **复合片键** ，例如由一个低基数的键和一个单调递增的键组成。

分片数据如何存储?

Chunk（块） 是 MongoDB 分片集群的一个核心概念，其本质上就是由一组 Document 组成的逻辑数据单元。每个 Chunk 包含一定范围片键的数据，互不相交且并集为全部数据，即离散数学中**划分**的概念。

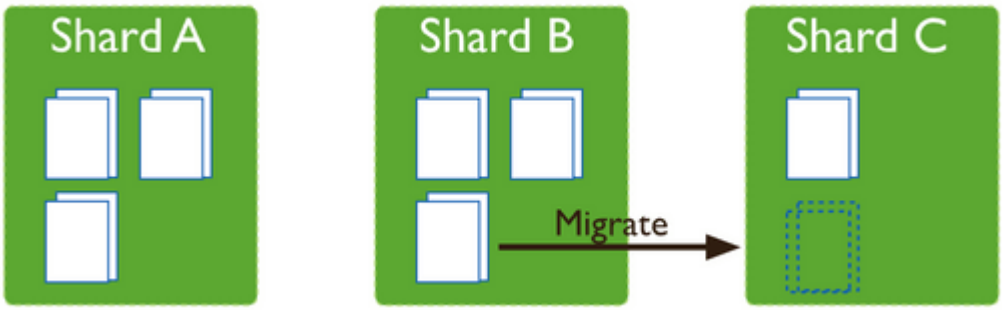
分片集群不会记录每条数据在哪个分片上，而是记录 Chunk 在哪个分片上一级这个 Chunk 包含哪些数据。

默认情况下，一个 Chunk 的最大值默认为 64MB（可调整，取值范围为 1~1024 MB。如无特殊需求，建议保持默认值），进行数据插入、更新、删除时，如果此时 Mongos 感知到了目标 Chunk 的大小或者其中的数据量超过上限，则会触发 **Chunk 分裂**。



数据的增长会让 Chunk 分裂得越来越多。这个时候，各个分片上的 Chunk 数量可能会不平衡。Mongos 中的 **均衡器(Balancer)** 组件就会执行自动平衡，尝试使各个 Shard 上 Chunk 的数量保持均衡，这个过程就是 **再平衡（Rebalance）**。默认情况下，数据库和集合的 Rebalance 是开启的。

如下图所示，随着数据插入，导致 Chunk 分裂，让 AB 两个分片有 3 个 Chunk，C 分片只有一个，这个时候就会把 B 分配的迁移一个到 C 分片实现集群数据均衡。



> Balancer 是 MongoDB 的一个运行在 Config Server 的 Primary 节点上(自 MongoDB 3.4 版本起)的后台进程，它监控每个分片上 Chunk 数量，并在某个分片上 Chunk 数量达到阈值进行迁移。

Chunk 只会分裂，不会合并，即使 chunkSize 的值变大。

Rebalance 操作是比较耗费系统资源的，我们可以通过在业务低峰期执行、预分片或者设置 Rebalance 时间窗等方式来减少其对 MongoDB 正常使用所带来的影响。

Chunk 迁移原理是什么？

关于 Chunk 迁移原理的详细介绍，推荐阅读 MongoDB 中文社区的一文读懂 MongoDB chunk 迁移 这篇文章。

学习资料推荐

- MongoDB 中文手册|官方文档中文版 （推荐）：基于 4.2 版本，不断与官方最新版保持同步。
- MongoDB 初学者教程——7 天学习 MongoDB ：快速入门。
- SpringBoot 整合 MongoDB 实战 - 2022 ：很不错的一篇 MongoDB 入门文章，主要围绕 MongoDB 的 Java 客户端使用进行基本的增删改查操作介绍。

参考

- MongoDB 官方文档（主要参考资料，以官方文档为准）：https://www.mongodb.com/docs/manual/
- 《MongoDB 权威指南》
- Indexes - MongoDB 官方文档：https://www.mongodb.com/docs/manual/indexes/
- MongoDB - 索引知识 - 程序员翔仔 - 2022：https://fatedeity.cn/posts/database/mongodb-index-knowledge.html

- MongoDB - 索引: https://www.cnblogs.com/Neeo/articles/14325130.html
- Sharding - MongoDB 官方文档：https://www.mongodb.com/docs/manual/sharding/

- MongoDB 分片集群介绍 - 阿里云文档：https://help.aliyun.com/document_detail/64561.html
- 分片集群使用注意事项 - - 腾讯云文档：https://cloud.tencent.com/document/product/240/44611

JavaGuide官方公众号
（微信搜索JavaGuide）

1、公众号后台回复“PDF”获取原创PDF面试手册

2、公众号后台回复“学习路线”获取Java学习路线最新版

3、公众号后台回复“开源”获取优质Java开源项目合集

4、公众号后台回复“八股文”获取Java面试真题+面经

最近更新2023/10/27 06:44

贡献者: guide,Guide,Erzbir,Mr.Hope

Copyright © 2025 Guide